PropOS Phase 1 — 开发启动手册

> 版本: v1.0
>
> 日期: 2026-04-04
>
> 适用对象: 项目负责人 / 开发者 / UI 设计师
>
> 前置文档: PRD.md · ARCH.md · DEV_UI_SYNC_GUIDE.md

目录

1. 当前状态与整体节奏
    - 1.1 当前状态（2026-04-04）
    - 1.2 整体节奏（14 天冲刺）
2. 环境准备（Day 0）
    - 2.1 后端工具链
    - 2.2 前端工具链
    - 2.3 其他工具
    - 2.4 项目目录结构确认
3. Step 1 — 后端脚手架
    - 3.1 初始化 Dart Shelf 项目
    - 3.2 配置 `pubspec.yaml`
    - 3.3 创建核心目录骨架
    - 3.4 实现顺序（Step 1 内部）
4. Step 2 — 数据库 Schema 迁移
    - 4.1 执行迁移文件
    - 4.2 编写迁移文件
    - 4.3 验证 Schema
5. Step 3 — M1 资产模块（后端）
    - 5.1 实现顺序
    - 5.2 关键实现要点
    - 5.3 单元测试检查
6. Step 4 — M2 合同模块（后端）
    - 6.1 实现顺序
    - 6.2 租金递增规则实现
    - 6.3 WALE 计算（必须含单元测试）
    - 6.4 预警定时任务

7. Step 5 — M3 财务模块（后端）
    7.1 实现顺序
    7.2 NOI 计算逻辑
    7.3 KPI 打分（必须含单元测试）
8. Step 6 — M4 工单模块（后端）
    8.1 工单状态机
    8.2 成本归口实现
    8.3 推送服务（Phase 1 Mobile）
9. Step 7 — M5 二房东模块（后端）
    9.1 行级数据隔离（最高安全优先级）
    9.2 审核流实现
    9.3 操作审计日志
10. Step 8 — 前端脚手架（uni-app + admin）
    10.1 初始化 uni-app 移动端项目
    10.2 初始化 admin PC 管理后台
    10.3 建立目录骨架
    10.4 MockData 覆盖要求
11. Step 9 — 开发与 UI 设计同步
    11.1 对齐日准备（Day 8，前端骨架完成后）
    11.2 双线并行（Day 9~11）
    11.3 Design Token 接入（Day 12）
12. Step 10 — 页面逐模块实现
    P0 页面（第一周必须）
    P1 页面（第二周）
    P2 页面（第三周）
    每个页面实现规范
13. Step 11 — 后端 API 对接（替换 MockData）
    13.1 API 对接流程
    13.2 API 对接顺序
14. Step 12 — 微信小程序精简版
    14.1 功能范围（极简）
    14.2 开发时机
    14.3 与 uni-app 移动端的分工
15. 验收检查清单
    15.1 后端核心计算验收
    15.2 功能验收

15.3 安全验收
15.4 性能验收
16. 关键约束与禁止事项
16.1 架构约束
16.2 开发/设计协作约束
16.3 Phase 1 范围红线

---

1. 当前状态与整体节奏

1.1 当前状态（2026-04-04）

项目	状态
`docs/PRD.md`	✅ 完成（v1.5）
`docs/ARCH.md`	✅ 完成（v1.0）
`docs/DEV_UI_SYNC_GUIDE.md`	✅ 完成
`backend/`	❌ 空目录，未初始化
`app/` + `admin/`	❌ 空目录，未初始化
数据库	❌ 未创建
Git 远端	✅ 已配置（github.com/SheltonWan/propos）

结论：文档阶段完成，进入第一行代码阶段。

1.2 整体节奏（14 天冲刺）

```
Day 0        : 环境准备
Day 1~2      : Step 1~2（后端脚手架 + DB Schema）
Day 3~5      : Step 3~5（M1/M2/M3 后端核心模块）
Day 6~7      : Step 6~8（M4/M5 后端 + 前端脚手架）
Day 8（对齐日）: Step 9（开发 × 设计对齐会议）
Day 9~11     : Step 10（前端页面骨架 + 设计并行）
Day 12~13    : Step 11（API 对接，去掉 MockData）
Day 14       : Step 12 + 验收清单走查
```

---

2. 环境准备（Day 0）

2.1 后端工具链

```
# 检查 Dart SDK（需要 3.x）
dart --version

# 如未安装，通过 Flutter SDK 获取，或使用官方安装包
# https://dart.dev/get-dart

# 检查 PostgreSQL（需要 15+）
psql --version

# 安装（macOS）
brew install postgresql@15
brew services start postgresql@15

# 创建开发数据库
createdb propos_dev
```

2.2 前端工具链

```
# 检查 Node.js（需要 18+）
node --version
npm --version

# 安装（推荐使用 nvm 管理版本）
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
nvm install 18
nvm use 18

# 安装 uni-app CLI（全局）
npm install -g @dcloudio/uni-cli
```

2.3 其他工具

```
# CAD 转换工具（.dwg → DXF → SVG/PNG）两步链路：
# 第一步：ODA File Converter（DWG → DXF，免费下载）
# 下载地址：https://www.opendesign.com/guestfiles/oda_file_converter
# 安装后验证（macOS 下通常在 /Applications/ODAFileConverter.app）
/Applications/ODAFileConverter.app/Contents/MacOS/ODAFileConverter --version

# 第二步：ezdxf（Python，DXF → SVG）
pip install "ezdxf[draw]"
ezdxf --version
```

2.4 项目目录结构确认

```
cd /Users/wanxt/app/propos
ls
# 期望看到: app/ admin/ backend/ docs/ pdfdocs/ scripts/
```

---

3. Step 1 — 后端脚手架

3.1 初始化 Dart Shelf 项目

```
cd backend
dart create -t server-shelf . --force
```

3.2 配置 `pubspec.yaml`

将 `backend/pubspec.yaml` 依赖节替换为:

```
name: propos_backend
description: PropOS Backend Server

environment:
  sdk: ^3.0.0

dependencies:
  shelf: ^1.4.2
  shelf_router: ^1.1.4
  postgres: ^3.3.0
  dart_jsonwebtoken: ^2.12.2
  crypto: ^3.0.3
  encrypt: ^5.0.3            # AES-256 证件号加密
  dotenv: ^4.2.0
  uuid: ^4.4.0
  freezed_annotation: ^2.4.4
  json_annotation: ^4.9.0
  excel: ^4.0.3              # Excel 导入
  csv: ^6.0.0

dev_dependencies:
  build_runner: ^2.4.12
  freezed: ^2.5.7
  json_serializable: ^6.8.0
  test: ^1.25.0
  lints: ^4.0.0
```

```
dart pub get
```

3.3 创建核心目录骨架

按 `ARCH.md §2` 的目录结构创建以下空文件，确保分层正确：

```
backend/
├── bin/server.dart
├── lib/
│   ├── config/
│   │   ├── app_config.dart
│   │   └── database.dart
│   ├── core/
│   │   ├── middleware/
│   │   │   ├── auth_middleware.dart
│   │   │   ├── rbac_middleware.dart
│   │   │   └── audit_middleware.dart
│   │   ├── errors/
│   │   │   ├── app_exception.dart
│   │   │   └── error_handler.dart
│   │   ├── pagination.dart
│   │   └── request_context.dart
│   ├── modules/
│   │   ├── auth/
│   │   ├── assets/
│   │   ├── contracts/
│   │   ├── finance/
│   │   ├── workorders/
│   │   └── subleases/
│   ├── router/
│   │   └── app_router.dart
│   └── shared/
│       ├── encryption.dart
│       └── validators.dart
├── test/
│   └── unit/
└── migrations/
```

3.4 实现顺序（Step 1 内部）

按以下顺序实现，每完成一个文件做一次 `git commit`：

优先级	文件	关键内容
1	`.env`（不提交 git）	`DB_URL`、`JWT_SECRET`、`AES_KEY`
2	`config/app_config.dart`	读取 `.env` 环境变量，提供全局配置常量
3	`config/database.dart`	PostgreSQL 连接池初始化（`postgres` 包 `Pool`）
4	`core/errors/app_exception.dart`	统一异常基类（`AppException`、`NotFoundEx`、`ForbiddenEx`）
5	`core/errors/error_handler.dart`	Shelf 中间件：捕获异常 → HTTP 状态码 + JSON 响应体

6	`core/request_context.dart`	请求上下文 PODO：`userId`、`role`、`boundContractId`（二房东专用，对应 JWT claim `bound_contract_id`）
7	`shared/encryption.dart`	AES-256-GCM 加解密工具（`encryptField` / `decryptField`）
8	`core/middleware/auth_middleware.dart`	JWT 验证 → 注入 `RequestContext` 到 `Request`
9	`core/middleware/rbac_middleware.dart`	接受 `List<UserRole>` 参数，校验当前角色权限
10	`core/middleware/audit_middleware.dart`	写入 `audit_logs` 表：操作人、路由、方法、时间
11	`bin/server.dart`	启动入口，管道组装：`rate_limit → error_handler → auth → rbac → audit → router`

> 安全提醒：`.env` 必须加入 `.gitignore`。`JWT_SECRET` 最短 32 字节随机字符串。`AES_KEY` 使用 256-bit（32 字节）随机密钥，存储在环境变量或密钥管理服务，不硬编码。

---

4. Step 2 — 数据库 Schema 迁移

4.1 执行迁移文件

按顺序执行 `docs/backend/MIGRATION_DRAFT_v1.7.md` 中列出的 SQL（详细字段定义见 `docs/backend/data_model.md`）：

```
psql propos_dev -f migrations/001_create_enums.sql
psql propos_dev -f migrations/002_create_users_and_audit.sql
psql propos_dev -f migrations/003_create_assets.sql
psql propos_dev -f migrations/004_create_contracts.sql
psql propos_dev -f migrations/005_create_finance.sql
psql propos_dev -f migrations/006_create_workorders.sql
psql propos_dev -f migrations/007_create_deposits.sql
psql propos_dev -f migrations/008_create_meter_readings.sql
psql propos_dev -f migrations/009_create_turnover_reports.sql
psql propos_dev -f migrations/010_create_subleases.sql
psql propos_dev -f migrations/011_create_kpi.sql
psql propos_dev -f migrations/012_create_import_batches.sql
psql propos_dev -f migrations/013_add_deferred_foreign_keys.sql
psql propos_dev -f migrations/014_seed_reference_data.sql
psql propos_dev -f migrations/015_create_departments.sql
psql propos_dev -f migrations/016_create_user_managed_scopes.sql
psql propos_dev -f migrations/017_create_kpi_targets_and_appeals.sql
```

4.2 编写迁移文件

将 `data_model.md` 完整 SQL 按照 `MIGRATION_DRAFT_v1.7.md` 的编号拆分写入各 `.sql` 文件。注意：

- `001_create_enums.sql`：所有 PostgreSQL ENUM 类型

- `002_create_users_and_audit.sql`：`users`、`audit_logs`、`job_execution_logs`、`refresh_tokens`
- `003_create_assets.sql`：`buildings → floors → floor_plans → units → renovation_records`
- `004_create_contracts.sql`：`tenants → contracts → contract_units → rent_escalation_phases → escalation_templates → alerts`
- `005_create_finance.sql`：`invoices → payments → expenses → kpi_*`
- `006_create_workorders.sql`：`suppliers → work_orders → work_order_photos`
- `010_create_subleases.sql`：`subleases`（含 `data_retention_until`）

> 循环依赖说明：`users.bound_contract_id` 引用 `contracts.id`，但 `contracts.created_by` 引用 `users.id`。解决方法：先建 `users` 表（不含 `bound_contract_id` 外键约束），建完 `contracts` 后用 `ALTER TABLE` 添加，即 `013_add_deferred_foreign_keys.sql`。

4.3 验证 Schema

```
psql propos_dev -c "\dt"              # 列出所有表
psql propos_dev -c "\dT+"             # 列出所有枚举类型
```

---

5. Step 3 — M1 资产模块（后端）

资产模块无外部依赖（仅依赖 `users`），优先实现，并作为后续模块的代码范式参考。

5.1 实现顺序

层	文件	核心内容
模型	`models/building.dart`	`@freezed Building`，含 `fromJson` / `toJson`
模型	`models/floor.dart`	`@freezed Floor`（含 `svgPath`、`pngPath`）
模型	`models/unit.dart`	`@freezed Unit`，三业态扩展字段用 `sealed class UnitExtra` 实现多态
模型	`models/renovation_record.dart`	`@freezed RenovationRecord`
Repository	`repositories/building_repository.dart`	CRUD SQL，全字段映射
Repository	`repositories/unit_repository.dart`	含按业态、状态过滤的查询方法
Service	`services/unit_service.dart`	状态计算（合同数据联动）、色块聚合统计
Service	`services/cad_import_service.dart`	调用 ODA File Converter（DWG→DXF）+ `ezdxf draw`（DXF→SVG）两步转换，保存路径到 DB
Service	`services/unit_import_service.dart`	解析 Excel，批量 upsert units（639 套）
Controller	`controllers/building_controller.dart`	HTTP 路由注册，调用 Service
Controller	`controllers/unit_controller.dart`	含 `GET /units?building=&type=&status=` 过滤

5.2 关键实现要点

单元状态计算（`unit_service.dart`）：

- `vacant`：无有效合同
- `leased`：有 `active` 状态合同，且到期日 > 90 天
- `expiring_soon`：有 `active` 合同，且到期日 ≤ 90 天
- `non_leasable`：`units.status` 手动标记，不随合同变化

CAD 导入流程：

```
POST /api/floors/:id/cad-import
  → 接收 multipart .dwg 文件
  → 保存到临时目录
  → 调用 ODA File Converter CLI（DWG→DXF）
  → 调用 ezdxf draw（DXF→SVG）
  → 上传 SVG/PNG 到文件存储
  → 更新 floors.svg_path / png_path
  → 返回预览 URL
```

Excel 批量导入（639 套）：

- 按业态提供三张模板（写字楼/商铺/公寓），`property_type` 字段区分
- 逐行校验：`unit_no` 唯一性、`floor_id` 存在性
- 使用事务包裹批量 INSERT，任意行失败整体回滚，返回错误行号

5.3 单元测试检查

```
dart test test/unit/unit_service_test.dart
```

---

6. Step 4 — M2 合同模块（后端）

6.1 实现顺序

层	文件	核心内容
模型	`models/tenant.dart`	`@freezed Tenant`，`certNoEncrypted` 字段标注加密注释
模型	`models/contract.dart`	`@freezed Contract`，含 `ContractStatus` 枚举
模型	`models/rent_escalation_rule.dart`	`sealed class RentEscalationRule`（6 子类）
Repository	`repositories/tenant_repository.dart`	读取时解密，写入时加密（调用 `encryption.dart`）
Repository	`repositories/contract_repository.dart`	含状态机转换 SQL 验证
Service	`services/wale_service.dart`	实现 WALE 公式（组合级、楼栋级、业态级）
Service	`services/rent_escalation_service.dart`	6 种递增类型计算引擎，返回未来各期租金序列
Service	`services/alert_service.dart`	预警触发逻辑（定时任务对接钩子）
Controller	全部	HTTP 路由

6.2 租金递增规则实现

递增规则使用 `sealed class` 实现多态，永远通过 `switch` 模式匹配处理：

```
// 6种递增类型
sealed class RentEscalationRule { ... }
class FixedPercentRule    extends RentEscalationRule { final double percent; ... }
class FixedAmountRule     extends RentEscalationRule { final double amountPerSqm; ... }
class SteppedRule         extends RentEscalationRule { final List<StepSegment> steps; ... }
class CpiLinkedRule       extends RentEscalationRule { final Map<int, double> cpiByYear; ... }
class EveryNYearsRule     extends RentEscalationRule { final int intervalYears; final double
percent; ... }
class PostFreeRentRule    extends RentEscalationRule { final double baseRent; final
RentEscalationRule? followUp; ... }
```

`rent_escalation_service.dart` 核心方法：

```
/// 给定合同和目标月份，返回当月应收租金（元/月）
Decimal calculateRentForMonth(Contract contract, DateTime targetMonth);

/// 返回合同全生命周期每月租金列表（用于未来租金预测 Excel）
List<MonthlyRent> generateRentSchedule(Contract contract);
```

6.3 WALE 计算（必须含单元测试）

```
dart test test/unit/wale_service_test.dart
```

测试用例须覆盖：

- 空合同列表（返回 0）
- 单合同
- 多合同混合业态
- 与手工 Excel 计算对比（误差 < 0.01 年为通过）

6.4 预警定时任务

`alert_service.dart` 提供 `checkAndCreateAlerts()` 方法，由 `bin/server.dart` 在启动时 + 每天 00:00 注册定时调用（使用 `dart:async` `Timer.periodic`）：

预警类型	检查时机	触发条件
租约到期	每天	`end_date - now() IN (90, 60, 30)` 天
租金逾期	每天	`invoice.due_date + X` 天未核销
月度到期汇总	每月 1 日 00:05	固定时间，汇总当月到期
押金退还	每天	`contract.end_date - now() = 7` 天

---

7. Step 5 — M3 财务模块（后端）

7.1 实现顺序

层	文件	核心内容
模型	`models/invoice.dart`	`@freezed Invoice`（含 `InvoiceStatus` 枚举）
模型	`models/expense.dart`	`@freezed Expense`（含支出类目枚举）
模型	`models/kpi_scheme.dart`	`@freezed KpiScheme`（含权重分配验证）
Repository	全部	标准 CRUD + 聚合查询
Service	`services/invoice_service.dart`	调用 `rent_escalation_service` 生成账单金额
Service	`services/noi_service.dart`	EGI - OpEx，按业态/楼栋分组
Service	`services/kpi_service.dart`	线性插值打分，聚合各指标，计算 KPI 总分
Controller	全部	含 `/api/noi/summary`、`/api/kpi/score`

7.2 NOI 计算逻辑

```
EGI = Σ 当月实收租金（按业态）- 空置损失估算 + 其他收入
OpEx = 水电公摊 + 外包物业费 + 维修费 + 保险 + 税金
NOI = EGI - OpEx
```

`noi_service.dart` 须支持按维度聚合：

- 全楼总 NOI
- 按楼栋（A座/商铺区/公寓楼）
- 按业态（office/retail/apartment）

7.3 KPI 打分（必须含单元测试）

```
dart test test/unit/kpi_service_test.dart
```

线性插值公式（每个指标独立计算）：

```
若 actual >= perfect_threshold  → score = 100
若 actual >= pass_threshold     → score = 60 + (actual - pass) / (perfect - pass) × 40
若 actual < pass_threshold      → score = max(0, actual / pass × 60)

KPI总分 = Σ(score_i × weight_i)
```

测试须覆盖：

- 全部 K01~K10 指标的满分、及格、不及格边界
- 两套不同权重方案总分计算一致性

8. Step 6 — M4 工单模块（后端）

8.1 工单状态机

```
submitted → approved → in_progress → pending_acceptance → completed
                                                ↓
                                        rejected / on_hold
```

允许的状态转换在 `work_order_service.dart` 中用 `Map<WorkOrderStatus, List<WorkOrderStatus>>` 枚举合法转换集合，非法转换抛 `InvalidStateTransitionException` 。

8.2 成本归口实现

完工时录入 `material_cost + labor_cost` ， `work_order_service.dart` 自动生成 `Expense` 记录关联：

- `unit_id` （精确到具体单元）
- `expense_type = 'repair'`
- `amount = material_cost + labor_cost`

此记录直接汇入 NOI 的 OpEx 计算。

8.3 推送服务（Phase 1 Mobile）

`push_service.dart` 封装 APNs / FCM 调用。桌面端（macOS/Windows）不发送推送，由前端轮询 `/api/alerts/unread` 。

---

9. Step 7 — M5 二房东模块（后端）

9.1 行级数据隔离（最高安全优先级）

`sublease_repository.dart` 所有查询方法必须强制注入 `boundContractId` 过滤：

```
Future<List<SubLease>> findByMasterContract(
  String masterContractId,
  RequestContext ctx,  // 含 ctx.boundContractId
) async {
  // 双重校验：参数 masterContractId == ctx.boundContractId
  // 防止 IDOR（不安全的直接对象引用）
  if (ctx.role == UserRole.subLandlord &&
      masterContractId != ctx.boundContractId) {
    throw ForbiddenException();
  }
  return pool.execute(
    'SELECT * FROM subleases WHERE master_contract_id = @id',
    {'id': masterContractId},
  );
```

```
}
```

9.2 审核流实现

```
子租赁提交 → review_status = 'pending_review'
                    ↓
    租务专员/运营管理层审核
          ↓                    ↓
    approved（生效）  rejected（附拒绝原因，返回二房东）
```

9.3 操作审计日志

`sublease_audit_logs` 表记录：操作人 ID、操作类型（create/update/delete/login/view）、操作前后数据 JSON（`JSONB`）、时间戳。对接 `audit_middleware.dart` 自动写入。

---

10. Step 8 — 前端脚手架（uni-app + admin）

10.1 初始化 uni-app 移动端项目

```
cd /Users/wanxt/app/propos/app
npm install
```

`app/package.json` 核心依赖：

```
{
  "dependencies": {
    "pinia": "^2.1.7",
    "luch-request": "^3.1.0",
    "wot-design-uni": "^1.3.0",
    "dayjs": "^1.11.10"
  },
  "devDependencies": {
    "@dcloudio/vite-plugin-uni": "^4.0.0",
    "typescript": "^5.4.0",
    "vite": "^5.2.0"
  }
}
```

10.2 初始化 admin PC 管理后台

```
cd /Users/wanxt/app/propos/admin
npm install
```

`admin/package.json` 核心依赖：

```
{
  "dependencies": {
    "vue": "^3.4.0",
```

```
    "vue-router": "^4.3.0",
    "pinia": "^2.1.7",
    "element-plus": "^2.7.0",
    "axios": "^1.7.0",
    "dayjs": "^1.11.10"
  },
  "devDependencies": {
    "@vitejs/plugin-vue": "^5.0.0",
    "typescript": "^5.4.0",
    "vite": "^5.2.0"
  }
}
```

10.3 建立目录骨架

uni-app 端（`app/src/`）：

```
app/src/
├── main.ts
├── App.vue
├── pages.json                    # uni-app 页面路由配置
├── manifest.json                 # 平台配置
├── uni.scss                      # 全局 CSS 变量 + Design Token
├── api/
│   ├── client.ts                 # luch-request 封装，apiGet/apiPost/apiPatch/apiDelete
│   ├── modules/                  # 按业务模块拆分的 API 函数
│   └── index.ts                  # 桶导出
├── constants/
│   ├── api_paths.ts             # API 路径常量
│   ├── business_rules.ts        # 业务规则常量
│   └── ui_constants.ts          # UI 展示常量
├── stores/                       # Pinia stores（defineStore setup 风格）
├── types/                        # TypeScript 接口定义
├── composables/                  # 可复用 Composition API 函数
├── pages/                        # 按 pages.json 结构组织页面
├── components/                   # 全局共享组件
└── static/                       # 静态资源
```

admin 端（`admin/src/`）：

```
admin/src/
├── main.ts
├── App.vue
├── api/
│   ├── client.ts                 # axios 封装，含 refresh subscriber queue
│   └── modules/
├── constants/
│   ├── api_paths.ts
│   ├── business_rules.ts
│   └── ui_constants.ts
├── stores/                       # Pinia stores
```

```
├── router/
│   └── index.ts                    # Vue Router 4，含 beforeEach 守卫
├── views/
│   ├── layout/                     # AppLayout（侧边栏 + 顶部栏）
│   ├── auth/                       # LoginView
│   ├── dashboard/
│   ├── assets/
│   ├── contracts/
│   ├── finance/
│   ├── workorders/
│   └── subleases/
├── components/                     # 全局共享组件
├── types/                          # TypeScript 接口定义
└── styles/
    └── variables.scss              # Element Plus 主题变量覆盖
```

10.4 MockData 覆盖要求

Mock 文件	覆盖内容
`mockDashboardData`	NOI、EGI、OpEx、出租率、WALE（三业态分别）、K01~K10 当前值
`mockBuildingData`	3 栋楼 × 多楼层 × 单元列表（含四种状态的单元）
`mockContractData`	10 份合同（覆盖全部 7 种状态机状态）
`mockInvoiceData`	20 条账单（pending/paid/overdue/waived 均覆盖）
`mockWorkOrderData`	8 条工单（覆盖全部 7 种工单状态）
`mockSubleaseData`	二房东视角：主合同 + 子租赁列表（含空置单元）

> 字段名一致性：MockData 字段命名必须与 `ARCH.md §3` 数据库列名（转 camelCase）完全一致，便于后续直接替换为 API 响应。

---

11. Step 9 — 开发与 UI 设计同步

> 本节是 `DEV_UI_SYNC_GUIDE.md` 的精华执行摘要，完整规范见原文。

11.1 对齐日准备（Day 8，前端骨架完成后）

开发者交付给设计师：

1. 骨架 App 录屏（5 段，每段覆盖一个主 Tab 流程）
2. 截图包（19 张，按页面编号，见 `DEV_UI_SYNC_GUIDE.md §5.1`）
3. 数据边界速查表（见 `DEV_UI_SYNC_GUIDE.md §5.3`）
4. 业务色彩语义定义（状态色不可被美化改变语义）

30 分钟对齐会议议程：

时间	内容
0~10 分钟	开发者演示骨架 App 操作流程
10~20 分钟	设计师提问：数据密度、交互边界
20~30 分钟	联合确认状态色彩语义、字体大小底线

11.2 双线并行（Day 9~11）

角色	任务
开发者	完善 Pinia 状态管理、Vue Router 守卫、表单验证、三态 UI（空/加载/错误）
设计师	Figma 高保真（P0 优先）、输出 Design Token 表格

11.3 Design Token 接入（Day 12）

设计师以表格提交 Token（颜色/字体/间距/圆角），开发者填入 `app/src/uni.scss` 和 `admin/src/styles/variables.scss` 。

固定语义色（设计师不得改变含义）：

CSS 变量	业务含义
`--color-success`	已租 → 绿色系
`--color-warning`	即将到期 → 黄/橙色系
`--color-danger`	空置 → 红色系
`--color-neutral` / `info`	非可租 → 中性灰

---

12. Step 10 — 页面逐模块实现

按优先级实现，每个页面遵循： `骨架（MockData） → 状态管理 → UI 精修` 。

P0 页面（第一周必须）

编号	页面	路由	设计复杂度
01	登录页	`/login`	★☆☆
02	Dashboard 总览	`/dashboard`	★★★
03	资产概览	`/assets`	★★☆
04	楼层平面图（热区）	`/assets/buildings/:bid/floors/:fid`	★★★
05	单元详情	`/assets/.../units/:uid`	★★☆
06	合同列表	`/contracts`	★☆☆
07	合同详情	`/contracts/:id`	★★★

P1 页面（第二周）

编号	页面	路由	设计复杂度
08	NOI 看板	`/finance`	★★★
09	账单列表	`/finance/invoices`	★★☆
10	工单列表	`/workorders`	★★☆
11	工单提报	`/workorders/new`	★★☆
12	KPI 看板	`/dashboard/kpi`	★★★
13	租金递增配置器	`/contracts/:id/escalation`	★★★

P2 页面（第三周）

编号	页面	路由	设计复杂度
14	二房东填报 Portal	`/sublease-portal`	★★☆
15	工单详情/审核	`/workorders/:id`	★☆☆
16	租客管理	`/tenants`	★☆☆
17	系统设置/用户管理	`/settings`	★☆☆
18	支出录入	`/finance/expenses/new`	★☆☆

每个页面实现规范

uni-app 端（`app/src/`）：

```
api/modules/<module>.ts        # API 函数
stores/<module>.ts              # Pinia store（defineStore setup 风格）
pages/<module>/
├── index.vue                 # 列表页
└── detail.vue                # 详情页
components/<Component>.vue      # 复用子组件
```

admin 端（`admin/src/`）：

```
api/modules/<module>.ts         # API 函数
stores/<module>.ts              # Pinia store
views/<module>/
├── IndexView.vue             # 列表页
└── DetailView.vue            # 详情页
components/<Component>.vue      # 复用子组件
```

三态 UI 强制要求（每个数据驱动页面）：

- `loading` → 显示加载骨屏 / Skeleton
- 空状态 → 显示提示文字 + 操作按钮
- 错误状态 → 显示错误信息 + 重试按钮

---

13. Step 11 — 后端 API 对接（替换 MockData）

13.1 API 对接流程

每个模块后端完成后，对应前端页面替换 MockData：

1. `api/modules/<module>.ts` 添加对应模块的 API 函数
2. Pinia store 的 action 改为调用 API 函数而非 Mock 数据
3. 在 `.env` 中配置 `VITE_API_BASE_URL`

```
// 替换前（MockData）
async function fetchDashboard() {
  loading.value = true
  list.value = mockDashboardData
  loading.value = false
}

// 替换后（真实 API）
async function fetchDashboard() {
  loading.value = true
  try {
    list.value = await apiGet<DashboardSummary>(API_PATHS.DASHBOARD)
  } catch (e) {
    error.value = e instanceof ApiError ? e.message : '加载失败，请重试'
  } finally {
    loading.value = false
  }
```

```
}
```

13.2 API 对接顺序

按模块依赖顺序，从无依赖模块开始：

```
Auth（登录/Token 刷新）
  → Assets（楼栋/楼层/单元）
  → Contracts（租客/合同）
  → Finance（账单/NOI/KPI）
  → WorkOrders（工单）
  → SubLeases（二房东）
```

---

14. Step 12 — 微信小程序精简版

微信小程序仅实现两个核心场景，不是主力开发方向：

14.1 功能范围（极简）

功能	说明
扫码报修	扫单元二维码 → 自动填入楼栋/楼层/单元 → 填写问题描述 + 上传 3 张照片 → 提交
工单状态查看	我提交的工单列表 → 点击查看当前状态（只读，无推送）

14.2 开发时机

待 uni-app 移动端 P1 页面稳定后再开始，复用同一后端 API，不单独维护数据层。

> 注：微信小程序已纳入 uni-app 多端编译范围，通过条件编译（ `#ifdef MP-WEIXIN` ）复用主体代码，仅小程序特有功能需单独处理。

14.3 与 uni-app 移动端的分工

功能	uni-app 移动端	微信小程序
完整工单流程	✅	❌
推送通知	✅ APNs/FCM/鸿蒙推送	❌
CAD 楼层快查	✅	❌
管理后台功能	✅	❌
扫码报修	✅	✅ 核心场景
查看工单状态	✅	✅ 只读

---

15. 验收检查清单

15.1 后端核心计算验收

验收项	标准	测试命令
WALE 计算	与 Excel 手算误差 < 0.01 年	`dart test test/unit/wale_service_test.dart`
租金递增规则	固定比例+阶梯混合计算结果与手算一致	`dart test test/unit/rent_escalation_test.dart`
NOI 计算	EGI - OpEx 与手工核算一致	`dart test test/unit/noi_service_test.dart`
KPI 打分	两套不同权重方案得分与手算一致	`dart test test/unit/kpi_service_test.dart`

15.2 功能验收

验收项	通过标准
CAD 平面图	.dwg 正确导入，热区色块实时联动合同状态
批量导入	639 套 Excel 模板成功导入，无数据错位
预警功能	模拟到期日 ≤30 天，系统 10 分钟内触发通知
工单闭环	移动端提报 → PC 派单 → 执行 → 验收，成本汇入 OpEx
二房东穿透	外部填报 → 审核 → 生效；行级隔离验证（A 二房东不可见 B 数据）

15.3 安全验收

安全项	检查方式
证件号加密	直接查数据库，`cert_no_encrypted` 列不可见明文
RBAC 中间件	用前线员工 Token 调用财务 API，应返回 403
行级隔离	二房东 A 的 Token 尝试查 B 的子租赁，应返回 403
JWT 过期	使用过期 Token 调用任意 API，应返回 401
API 响应脱敏	证件号在 API 响应中只显示后 4 位

15.4 性能验收

性能项	标准
Dashboard 加载	< 3 秒（含 NOI + WALE + KPI 三个聚合查询）
账单批量生成	639 条 < 30 秒
楼层热区图渲染	SVG 首次渲染 < 2 秒
并发用户	50 并发下各接口响应时间 < 1 秒（k6 压测）

---

16. 关键约束与禁止事项

16.1 架构约束

约束	说明
所有 API 必须经过 RBAC	不允许跳过 `rbac_middleware` 的 API 端点
行级隔离在 Repository 层强制	`sublease_repository` 所有方法内部必须从 `ctx.boundContractId` 提取隔离条件，不依赖调用方主动传入
证件号落库前必须加密	调用 `encryption.encryptField()`，任何 Service 层不存储明文
API 响应默认脱敏	Controller 序列化时，`certNo` 只输出 `certNoHint`（后4位）
核心计算必须有单元测试	WALE、NOI、KPI、租金递增，无测试不视为完成

16.2 开发/设计协作约束

禁止行为	原因
设计师直接修改 Vue 代码	破坏代码分层约定
开发者绕过 Figma 自行美化	产生设计漂移
以 Figma 截图替代真机/浏览器验收	Figma 不模拟真实字体渲染和屏幕密度
改变状态色彩的业务语义	`expiring_soon` 必须是黄/橙色，这是业务约束非美观偏好

16.3 Phase 1 范围红线

以下功能不在 **Phase 1 范围内**，需求讨论中提起时明确拒绝：

- 租户自助门户（Phase 2）
- 在线付款（Phase 2）
- 门禁集成（Phase 2）
- 电子签章（Phase 2）
- 外包物业账号（Phase 2）

> 文档更新：每完成一个 Step，在本文对应章节顶部标注 ✅ `DONE - YYYY-MM-DD`，保持进度可见。